1. Software Test Guide

FUNCTIONAL

Document Control	
Author(s)	Michael Gardner
Version	4.1.0
Status	Released
Status Date	2025-12-18
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	library
Library Role	utility
Application Role	N/A
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Software Test Guide ... 1
2. Introduction ... 1
   - 2.1. Purpose ... 1
   - 2.2. Scope ... 1
   - 2.3. References ... 1
3. Test Strategy ... 1
   - 3.1. Test Categories ... 1
   - 3.2. Testing Philosophy ... 1
4. Test Organization ... 1
   - 4.1. Directory Structure ... 1
   - 4.2. Naming Conventions ... 2
   - 4.3. Build/Test Projects ... 2
5. Test Framework ... 2
   - 5.1. Framework Overview ... 2
   - 5.2. Framework API ... 2
   - 5.3. Usage Pattern ... 2
6. Test Execution ... 3
   - 6.1. Running All Tests ... 3
   - 6.2. Running Specific Suites ... 3
   - 6.3. Expected Output ... 3
   - 6.4. SPARK Verification ... 3
7. Test Details ... 4
   - 7.1. Unit Tests ... 4
     - 7.1.1. Result Tests (`test_result.adb`) ... 4
     - 7.1.2. Option Tests (`test_option.adb`) ... 4
     - 7.1.3. Either Tests (`test_either.adb`) ... 4
     - 7.1.4. Try Tests (`test_try.adb`, `test_try_option.adb`, `test_try_map_to_result.adb`) ... 4
     - 7.1.5. Scoped Tests (`test_scoped.adb`) ... 4
   - 7.2. SPARK Tests ... 4
     - 7.2.1. SPARK Instantiations (`spark_instantiations.ads`) ... 4
8. Writing New Tests ... 4
   - 8.1. Test Template ... 4
   - 8.2. Adding to Build Configuration ... 5
   - 8.3. Test Doubles / Mock Patterns ... 5
9. Traceability ... 5
   - 9.1. Requirements to Tests ... 5
   - 9.2. Layer Coverage ... 6
10. Test Maintenance ... 6
    - 10.1. When to Update ... 6
    - 10.2. Quality Guidelines ... 6
    - 10.3. Continuous Integration ... 6
11. Appendices ... 7
    - 11.1. Change History ... 7

2. Introduction

2.1. Purpose

This Software Test Guide (STG) describes the testing strategy, test organization, execution procedures, and maintenance guidance for the **Functional** library.

2.2. Scope

This document covers:

- test categories and their purposes,
- test directory structure and naming conventions,
- test framework usage,
- test execution commands,
- traceability between requirements and tests, and
- guidelines for writing new tests.

2.3. References

- Software Requirements Specification (SRS).
- Software Design Specification (SDS).

3. Test Strategy

3.1. Test Categories

Category	Location	Purpose	Target
Unit Tests	`test/unit/`	Test individual packages in isolation.	≥ 90% coverage
SPARK Tests	`test/spark/`	Verify SPARK-compatible instantiations.	100% legality
Integration Tests	`test/integration/`	Cross-package workflows.	Key workflows

See CHANGELOG for current test counts per release.

3.2. Testing Philosophy

- **Test behavior, not implementation** — tests verify observable behavior.
- **One assertion per concept** — each test validates one logical concept.
- **Descriptive names** — test names describe what is being tested.
- **No mocking core types** — Result, Option, Either are tested directly.
- **SPARK verification** — domain types are verified with GNATprove.

4. Test Organization

4.1. Directory Structure

```
test/
├── common/                         # Shared test infrastructure
│   ├── test_framework.ads          # Framework spec
│   └── test_framework.adb          # Framework impl
├── config/                         # Test configuration
│   └── functional_tests_config.ads # Auto-generated by Alire
├── unit/                           # Unit test suites
│   ├── test_result.adb             # Result tests
│   ├── test_option.adb             # Option tests
│   ├── test_either.adb             # Either tests
```

```
│   ├── test_try.adb                # Try tests
│   ├── test_try_option.adb         # Try Option tests
│   ├── test_try_map_to_result.adb  # Map_To_Result tests
│   ├── test_try_map_to_result_with_param.adb
│   ├── test_scoped.adb             # Scoped tests
│   └── unit_runner.adb             # Unit test runner
├── spark/                          # SPARK verification tests
│   ├── spark_instantiations.ads    # SPARK-compatible instantiations
│   └── spark_instantiations.adb
├── functional_test.gpr             # Test GPR project
└── alire/                          # Alire dependencies (generated)
```

4.2. Naming Conventions

Element	Convention	Example
Runner	`unit_runner.adb`	`unit_runner.adb`
Test file	`test_<module>.adb`	`test_result.adb`
Test package	`Test_<Module>`	`Test_Result`
Test procedure	`Test_<Operation>_<Scenario>`	`Test_Map_Success`

4.3. Build/Test Projects

Project	Purpose
`functional_test.gpr`	Builds all test executables; extends `functional.gpr`.

5. Test Framework

5.1. Framework Overview

The Functional library uses a lightweight custom test framework (`test/common/test_framework.ads`).

5.2. Framework API

```
package Test_Framework is
   procedure Register_Results
     (Suite_Name   : String;
      Test_Count   : Natural;
      Passed_Count : Natural);

   function All_Passed return Boolean;

   procedure Report_And_Exit;
end Test_Framework;
```

5.3. Usage Pattern

```
with Test_Framework;

procedure Test_My_Module is
   Test_Count   : Natural := 0;
   Passed_Count : Natural := 0;

   procedure Assert (Condition : Boolean; Name : String) is
   begin
      Test_Count := Test_Count + 1;
```

```
      if Condition then
         Passed_Count := Passed_Count + 1;
         Put_Line ("[PASS] " & Name);
      else
         Put_Line ("[FAIL] " & Name);
      end if;
   end Assert;

begin
   Assert (Is_Ok (Result), "Ok creates success");

   Test_Framework.Register_Results
     ("My_Module", Test_Count, Passed_Count);
end Test_My_Module;
```

6. Test Execution

6.1. Running All Tests

```
make test-all
```

Or manually:

```
alr build
./bin/test/unit_runner
```

6.2. Running Specific Suites

```
# Run individual test
./bin/test/test_result

# Run unit tests only
make test-unit
```

6.3. Expected Output

```
========================================
Test Summary: Functional.Result
========================================
Total tests: 84
Passed: 84
Failed: 0
========================================
All tests passed!
```

6.4. SPARK Verification

```
# Run SPARK proof
make spark-prove

# Run SPARK legality check only
make spark-check
```

7. Test Details

7.1. Unit Tests

7.1.1. Result Tests (`test_result.adb`)

Tests all Result operations: constructors (Ok, Error), queries (Is_Ok, Is_Error, Value, Get_Error), transformations (Map, Map_Error, And_Then, Or_Else), and extraction (Unwrap_Or, Match).

7.1.2. Option Tests (`test_option.adb`)

Tests all Option operations: constructors (Some, None), queries (Is_Some, Is_None, Value), transformations (Map, And_Then, Or_Else, Filter), and extraction (Unwrap_Or, Match).

7.1.3. Either Tests (`test_either.adb`)

Tests all Either operations: constructors (Left, Right), queries (Is_Left, Is_Right, Left_Value, Right_Value), and transformations (Map_Left, Map_Right, Fold).

7.1.4. Try Tests (`test_try.adb`, `test_try_option.adb`, `test_try_map_to_result.adb`)

Tests exception bridging: success path (no exception), exception path (caught), exception mapping (correct error kind), and default error kind (unmapped exceptions).

7.1.5. Scoped Tests (`test_scoped.adb`)

Tests RAII guards: normal cleanup path, exception in guarded block, and exception in Release (suppressed).

7.2. SPARK Tests

7.2.1. SPARK Instantiations (`spark_instantiations.ads`)

Verifies that Result, Option, and Either can be instantiated under SPARK_Mode On:

```
pragma SPARK_Mode (On);

package SPARK_Instantiations is
   type Error_Kind is (E1, E2);

   package Int_Result is new Functional.Result (Integer, Error_Kind);
   package Int_Option is new Functional.Option (Integer);
end SPARK_Instantiations;
```

8. Writing New Tests

8.1. Test Template

```
pragma Ada_2022;
--  ======================================================================
--  Test_New_Feature
--  ======================================================================
--  Copyright (c) 2025 Michael Gardner, A Bit of Help, Inc.
--  SPDX-License-Identifier: BSD-3-Clause
--  Purpose: Unit tests for New_Feature.
--  ======================================================================

with Ada.Text_IO;    use Ada.Text_IO;
with Ada.Command_Line;
with Functional.New_Feature;
with Test_Framework;
```

```
procedure Test_New_Feature is
   Test_Count   : Natural := 0;
   Passed_Count : Natural := 0;

   procedure Assert (Condition : Boolean; Test_Name : String) is
   begin
      Test_Count := Test_Count + 1;
      if Condition then
         Passed_Count := Passed_Count + 1;
         Put_Line ("[PASS] " & Test_Name);
      else
         Put_Line ("[FAIL] " & Test_Name);
      end if;
   end Assert;

begin
   --  Tests here

   Test_Framework.Register_Results
     ("New_Feature", Test_Count, Passed_Count);

   if Passed_Count = Test_Count then
      Ada.Command_Line.Set_Exit_Status (Ada.Command_Line.Success);
   else
      Ada.Command_Line.Set_Exit_Status (Ada.Command_Line.Failure);
   end if;
end Test_New_Feature;
```

8.2. Adding to Build Configuration

Add new test to `functional_test.gpr`:

```
for Main use (
   "unit_runner.adb",
   "test_new_feature.adb"  --  Add new test
);
```

8.3. Test Doubles / Mock Patterns

Mocking is generally not needed for Functional library tests:

- Core types are pure (no I/O).
- Try tests use controlled exceptions.
- Scoped tests use test resources.

9. Traceability

9.1. Requirements to Tests

Requirement	Test Coverage
FR-R01..R12	`test_result.adb` — constructors, queries, transformations, extraction
FR-O01..O10	`test_option.adb` — constructors, queries, transformations, extraction
FR-E01..E08	`test_either.adb` — constructors, queries, transformations
FR-T01..T07	`test_try.adb`, `test_try_map_to_result.adb`, `test_try_map_to_result_with_param.adb`
FR-S01..S04	`test_scoped.adb` — guard lifecycle, exception suppression

Requirement	Test Coverage
NFR-07	`spark_instantiations.ads` — SPARK legality verification

9.2. Layer Coverage

Module Family	Representative Test Areas
Core Types (SPARK)	`test_result.adb`, `test_option.adb`, `test_either.adb`, `spark_instantiations.ads`
Exception Bridges	`test_try.adb`, `test_try_option.adb`, `test_try_map_to_result*.adb`
RAII Guards	`test_scoped.adb`

10. Test Maintenance

10.1. When to Update

- **New operation added**: add tests for the new operation.
- **Behavior changed**: update affected tests.
- **Bug fixed**: add a regression test.
- **API signature changed**: update test instantiations.

10.2. Quality Guidelines

- Each public operation should have at least one test.
- Test both success and failure paths.
- Test edge cases (empty values, boundary conditions).
- Keep tests independent (no shared mutable state).

10.3. Continuous Integration

Tests run automatically on:

- pull request creation,
- push to main branch, and
- release tag creation.

11. Appendices

11.1. Change History

Version	Date	Author	Changes
4.1.0	2025-12-18	Michael Gardner	Add test warning suppressions; migrate to Typst format.
4.0.0	2025-12-12	Michael Gardner	Add Map_To_Result and Map_To_Result_With_Param tests.
3.0.0	2025-11-30	Michael Gardner	Initial test framework.